



MASTER OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

Assignment #3

Arif Kunto Wibisono

StudentID: *a1915248*

Email: *a1915248@adelaide.edu.au*

Course: *Applied Artificial Intelligence and Machine Learning 2*

Contents

Summary of Code Implementation	1
Introduction	1
Model Architecture	1
Forward Propagation	4
Backward Propagation	5
Training Loop	6
Evaluation	7
Challenges Faced and Solutions Applied	8
Number of Layers	8
Encoder Block Hyperparameter	8
Dropout	9
Question 1 - Attention	11
Answer:	11
Question 2 - Analyzing Model Performance	12
Answer:	12
References	15
Acknowledgement of Generative AI Usage	15

Summary of Code Implementation

Introduction

This project implements a simplified Transformers model from scratch to perform sentiment analysis on IMDb dataset. The project emphasises on the core building blocks of Transformer-based architectures, such as self-attention and positional encoding. As required in this assignment, no prebuilt Transformers libraries (e.g., Hugging Face Transformers) can be used. However, this project is permitted to use torch library. Hence, the process uses GPU for faster processing. For the summary of code implementation, we used notebook Assignment_3_v1-Arif-a1915248.ipynb

Model Architecture

Figure 1 shows the model architecture

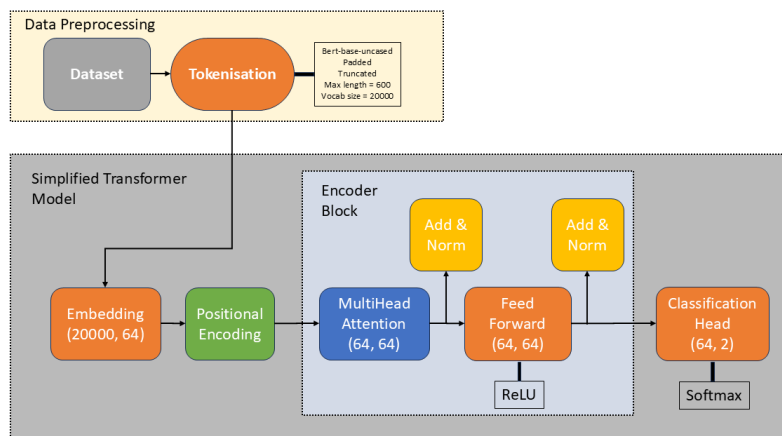


Figure 1: Model Architecture

As per the assignment requirement, the model architecture is:

- Input Embedding Layer:
 - Convert input text to token embeddings using a simple word embedding layer.
 - Use torch.nn.Embedding or implement on our own.
- Positional Encoding:
 - Add positional encodings to input embeddings to incorporate sequential information.
- Self-Attention Mechanism:
 - Implement scaled dot-product attention:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$
 - Use multi-head attention for richer feature representations.

- Feed Forward Network:
 - Add a two-layer feedforward network with ReLU activation.
- Classification Head:
 - Use a fully connected layer to map the output of the Transformer to a binary label (positive or negative).
- Architecture Summary:
 - Input → Embedding → Positional Encoding → Multi-Head Attention → Feed-forward → Classification Head → Output

In this project, we used IMDB dataset, which contains 25,000 rows of train datasets as well as 25,000 test datasets. To understand the dataset and guide us build a proper model, we conduct Exploratory Data Analysis. First, we calculated the distribution of character length on the train and test dataset. Figure 2 shows the character length comparison

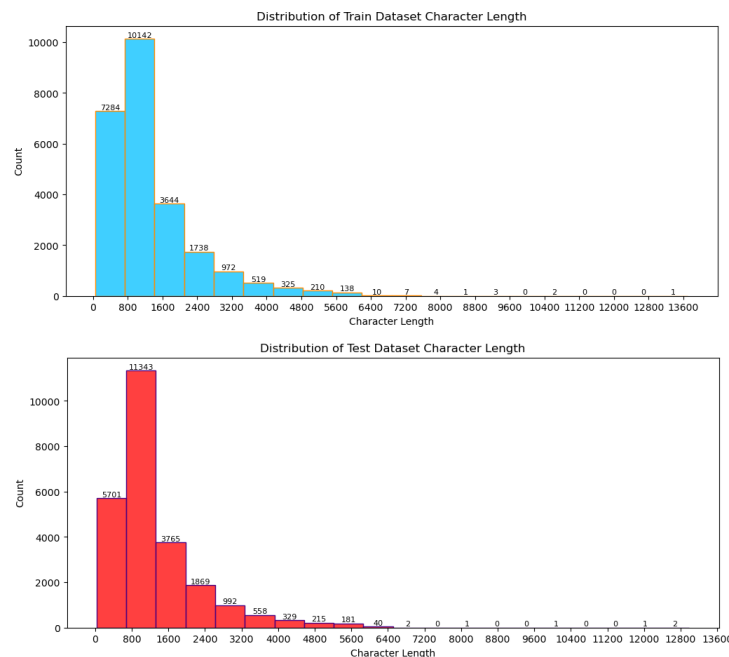


Figure 2: Character Length Comparison

From figure 2, we know that both the distribution centered on approximately 800-1200 characters. Next, we tried to understand the words count across the dataset. We then split the reviews to acquire the words in every datapoint, then count the number of words in it. Figure 3 displays the word count comparison

From figure 3, we know that for both of the dataset, the word count is centered on around 110 - 250 words. The label distribution is nearly identical, means that the dataset is balanced. To get a good approximation of number of sequence length in tokenisation, we calculate the 95th percentile of the train and test dataset, and get 598 and 582. It means that only 5% of the review that has word count above 598 (for train set) and 582 (for test set). With this result, we decided to set the sequence length of 600. That means, if a datapoint contains word count above 600, it will be truncated.

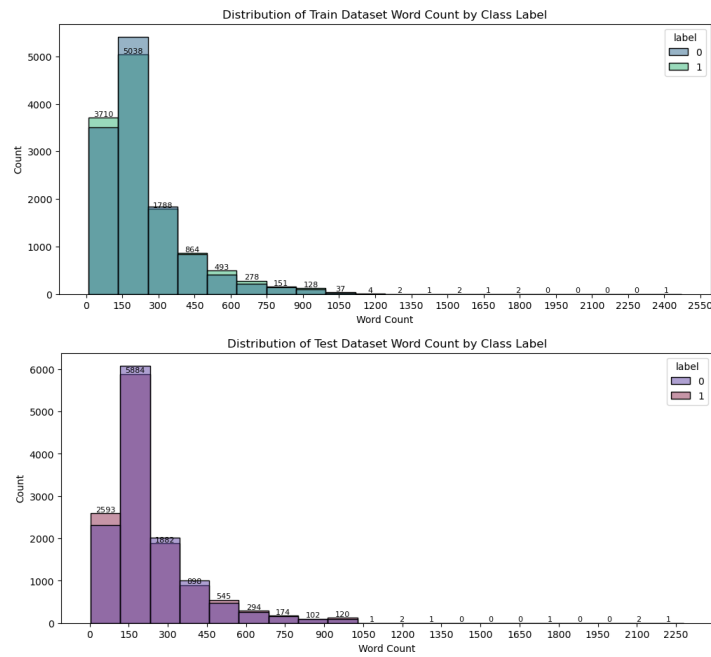


Figure 3: Word Count Comparison

We build our own tokenisation function. The maximum sequence length for the token is 600. Section [Forward Propagation](#) explains in detail about tokenisation. From this process, the token counts are centered around 130 - 150 tokens. Figure 4 displays the token count comparison.

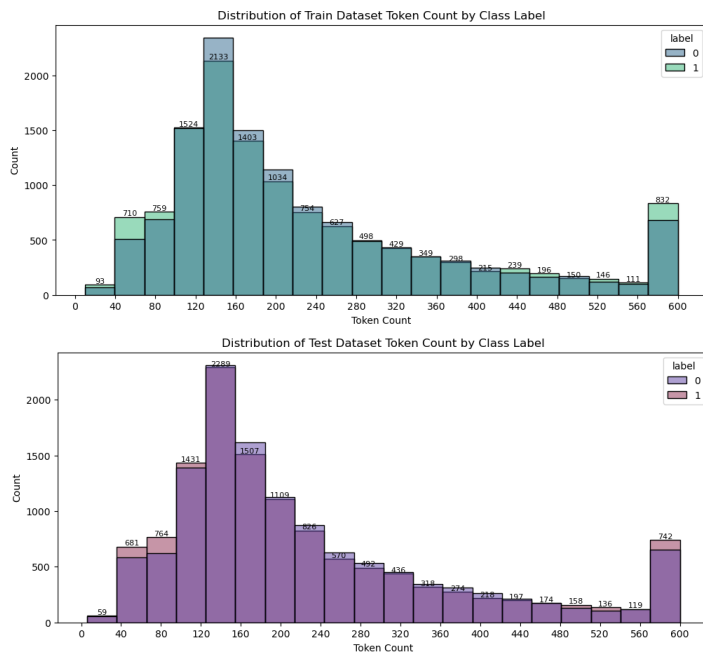


Figure 4: Token Count Comparison

In this project, we used this setting for the encoder block:

- num_layers: 4

- `d_model`: 64
- `num_heads`: 4
- `d_ff`: 256
- `input_vocab_size`: 20000
- `max_seq_length`: 600

As the regularisation technique, we implement dropout of 0.1.

Forward Propagation

In forward propagation, we have four operations: tokenisation, embedding, attention computation, and feedforward operations.

- Tokenisation

Tokenisation is the process of dividing up the input text into subunits, called tokens. These tokens are then converted into numerical representations (ID tokens) that machine learning can process (Grefenstette, 1999). In this project, we implement a custom word-level tokenisation function.

The custom tokenisation function operates in two stages. Firstly, the `tokenise_words` function applies regular expression to extract individual words from the raw text. It removes `<br>`, converts all text to lowercase, and identify boundaries, removing punctuation and HTML artifacts. This produces a list of lowercase word tokens.

Secondly, these word tokens are mapped to IDs using vocabulary dictionary. The vocabulary is programmed by counting word frequencies across the entire training corpus and retaining the 20,000 most common words. The out-of-vocabulary in this dictionary is 2.29% for train dataset and 2.97% for test dataset. Next, sequences are padded or truncated to a maximum length of 600 tokens.

- Embedding

Embedding represents text in form of vectors. Specifically, embedding is a vector representation that map discrete tokens into vector space, enabling neural networks to capture the semantic relationships (Verma and Khandelwal, 2019).

In this project, we implemented embedding using PyTorch's `torch.nn.Embedding` layer. The embedding layer is initialised with dimensions (`input_vocab_size`, `d_model`), where `input_vocab_size` = 20,000 and `d_model` = 64 (the embedding dimension). To prevent the model from learning embeddings for padding tokens, the `padding_idx` parameter is set to 0, which forces the [PAD] token's embedding to remain zero vector and excludes it from gradient updates.

- Attention Computation

Attention function can be described as a dot product of a query and a set of key-value pairs to an output, where the query, keys, and output are all vectors. In this project, we used scaled dot-product attention. The formula to calculate attention is below:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

Instead of single attention, we performed a multi-head attention. Multi-head attention allows the model to jointly attend to information from different representation subspaces at different positions (Vaswani et al., 2017).

- Feedforward Operations

Feedforward network are fully-connected neural network, applied separately to each position. This operations provide additional non-linear transformations, with ReLU (Rectified Linear Unit) activation in between (Vaswani et al., 2017).

In this project, the feedforward network consist of two torch.nn.LayerNorm. Firstly, it applies a linear transformation from 64 dimensions to 256 dimensions. It followed by ReLU activation functions to introduce non-linearity. The expansion to 256 dimensions in the hidden layer increases the model's capacity to learn complex patterns. Finally, it applies linear transformation from 256 dimensions to 64 dimensions to match the model architecture.

Table 1 summarise the spatial dimension changes in forward propagation.

Table 1: Spatial Dimension Changes

Step	Operation	Input Shape	Output Shape
1	Input (token IDs)	(batch_size, 600)	(batch_size, 600)
2	Embedding Lookup	(batch_size, 600)	(batch_size, 600, 64)
3	Embedding Scaling ($\times \sqrt{d_{\text{model}}}$)	(batch_size, 600, 64)	(batch_size, 600, 64)
4	Positional Encoding Addition	(batch_size, 600, 64)	(batch_size, 600, 64)
5	Dropout	(batch_size, 600, 64)	(batch_size, 600, 64)
6	Multi-Head Attention (4 heads)	(batch_size, 600, 64)	(batch_size, 600, 64)
7	Layer Normalization 1	(batch_size, 600, 64)	(batch_size, 600, 64)
8	Feedforward Layer 1 (64 $\rightarrow$ 256)	(batch_size, 600, 64)	(batch_size, 600, 256)
9	ReLU Activation	(batch_size, 600, 256)	(batch_size, 600, 256)
10	Feedforward Layer 2 (256 $\rightarrow$ 64)	(batch_size, 600, 256)	(batch_size, 600, 64)
11	Layer Normalization 2	(batch_size, 600, 64)	(batch_size, 600, 64)
12	Encoder Blocks (4 layers total)	(batch_size, 600, 64)	(batch_size, 600, 64)
13	Masked Mean Pooling	(batch_size, 600, 64)	(batch_size, 64)
14	Classification Head (64 $\rightarrow$ 2)	(batch_size, 64)	(batch_size, 2)
15	Softmax Activation	(batch_size, 2)	(batch_size, 2)

Backward Propagation

For backward propagation, we used binary cross-entropy loss to compute the gradients and applied Adam as optimisation techniques.

- Binary Cross Entropy Loss

Binary Cross Entropy (BCE) is a loss function used for binary classification tasks. It calculates the difference between the actual class labels (0 or 1) and the predicted probabilities output by the model. The formula for BCE loss function is:

$$\text{BCE} = -\frac{1}{N} \sum_{i=1}^N [y_i \log(p_i) + (1 - y_i) \log(1 - p_i)]$$

In this project, we use PyTorch's `torch.nn.BCELoss()` with the model's output probability of the positive class. During each training loop, the loss is computed as the mean in the batch and then backpropagated to update model parameters. The loss value is accumulated across all batches to compute the epoch-level average loss, then plotted to visualise the model's learning progress across epochs.

- Adam Optimiser

The Adam (Adaptive Moment Estimation) optimiser is an adaptive learning rate algorithm that combines the advantages of AdaGrad and RMSprop. Several advantages of using Adam are: the magnitudes of parameter updates are invariant to rescaling of the gradient, its stepsizes are approximately bounded by the stepsize hyperparameter, it does not require a stationary objective, it works with sparse gradients, and it naturally performs a form of step size annealing (Kingma and Ba, 2014)

The general formula of Adam optimiser are as follows:

$$\begin{aligned} g_t &\leftarrow \nabla_{\theta} f_t(\theta_{t-1}) && \text{(Get gradients w.r.t. stochastic objective at timestep } t) \\ m_t &\leftarrow \beta_1 \cdot m_{t-1} + (1 - \beta_1) \cdot g_t && \text{(Update biased first moment estimate)} \\ v_t &\leftarrow \beta_2 \cdot v_{t-1} + (1 - \beta_2) \cdot g_t^2 && \text{(Update biased second raw moment estimate)} \\ \hat{m}_t &\leftarrow m_t / (1 - \beta_1^t) && \text{(Compute bias-corrected first moment estimate)} \\ \hat{v}_t &\leftarrow v_t / (1 - \beta_2^t) && \text{(Compute bias-corrected second raw moment estimate)} \\ \theta_t &\leftarrow \theta_{t-1} - \alpha \cdot \hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon) && \text{(Update parameters)} \end{aligned}$$

where m_t is the exponential moving average of gradients (first moment), v_t is the exponential moving average of squared gradients (second moment), g_t is the gradient at step t , and β_1 and β_2 are decay rates (typically 0.9 and 0.999), α is the learning rate, and ϵ is a small constant for numerical stability. This adaptive mechanism allows Adam to converge faster.

In this project, the optimiser is implemented as `torch.optim.Adam(encoder.parameters(), lr=0.001)`. During each training step, `optimiser.zero_grad()` clears the accumulated gradients, `loss.backward()` computes new gradients via backpropagation, and `optimiser.step()` updates all model parameters according to the Adam algorithm.

Training Loop

For the training loop, we use a learning rate of 0.001 and batch size of 32. The training loader is shuffled to randomise the order each epoch and prevent overfitting to the same sequence. Although the assignment does not explicitly require dropout or other regularisation techniques, we implemented dropout of 0.1 to improve model generalisation. This technique is discussed in detail in [Dropout](#) section.

Inside the training loop, we also execute validation. The validation is performed on the training loader. Therefore, both training loss and validation loss are computed on the same data. Training completes in approximately 20 minutes. Training and validation losses are plotted across all epochs.

Evaluation

The testing phase uses a held-out test dataset that the model has not seen during training. The model achieves 0.8664 accuracy. The corresponding notebook for this is `Assignment_3_v1-Arif-a1915248.ipynb`. Figure 5 displays the learning curve and confusion matrix for this implementation.

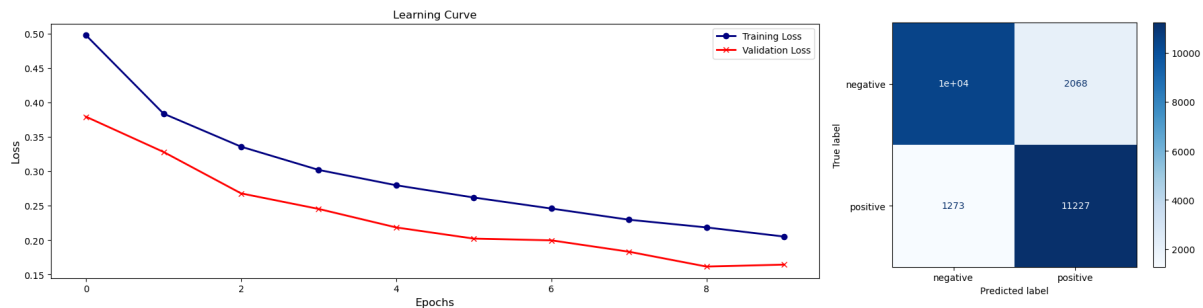


Figure 5: Learning Curve and Confusion Matrix v1

As shown in the figure, the model exhibits no signs of overfitting or underfitting. The decreasing loss curve suggests potential for further improvement beyond 10 epochs. The confusion matrix shows that the model confidently predicts the labels of the IMDB dataset. Table 2 shows the classification report from this model.

Table 2: Classification Report

Class	Precision	Recall	F1-Score	Support
negative	0.89	0.83	0.86	12500
positive	0.84	0.90	0.87	12500
Accuracy			0.87	25000
Macro Avg	0.87	0.87	0.87	25000
Weighted Avg	0.87	0.87	0.87	25000

From this table, we can conclude that the model achieve a good performance from precision, recall, and F1-score matrices.

Challenges Faced and Solutions Applied

The challenges we faced and solutions we applied in this project are as follows:

Number of Layers

Initially, we tested different number of encoder layers (2, 4, 6, and 8) to understand their effect on model's performance. With fewer layer (2 layers), the model has achieved lower accuracy due to lower representational depth, resulting in shallow feature extraction that struggles to capture complex linguistic patterns. On the other hand, more layers (6-8 layers) can provide expressiveness but has risk of overfitting, given the relatively small dataset size, leading to a weaker generalisation. In essence, more layers require more training data to learn meaningful representations. With 4 encoder layers, we achieved optimal performance, balancing the models's capacity to learn sentiment patterns while maintaining a reasonable generalisation to the test dataset.

Encoder Block Hyperparameter

The encoder block has several hyperparameter, which is `d_model`, `num_heads`, `d_ff`, `input_vocab_size`, and `max_sequence_length`. We have tested several combinations of these hyperparameters.

- **d_model**: the embedding dimension controls the width of token vector representations in the model. In this project, we set `d_model = 64`. A larger `d_model` can increase the model's capacity to learn richer semantic representations, but at the same time increase the memory consumption and computational cost. Conversely smaller `d_model` limit the power to learn the dataset features. `d_model = 64` is sufficient to capture the semantic features of the IMDB dataset, while maintain computational cost and achieve good performance.
- **num_heads**: the number of heads determines how many attention heads operate in multi-head attention mechanism. A more attention heads (8 or 16) allow the model to simultaneously attend to different representation subspaces and capture different types of relationship within the text. More heads require more parameters and computational resources. With `num_heads = 4` and `d_model = 64`, each head operates on 16 representation subspaces ($64/4$), providing sufficient capability to learn diverse pattern of sentiment without excessive computational power.
- **d_ff**: specifies the hidden dimension of the position-wise feedforward network within each encoder block. We set `d_ff = 256`, which expands from `d_model = 64` in the first layers, and transforms it back to 64 dimension in the second layer. This expansion increases the model's non-linear transformation. A larger `d_ff` (512 or 1024) increases the parameters and training time to boost the expressiveness. With `d_ff = 256` (4x embedding dimension), it balances the computational efficiency with adequate expressiveness for IMDB dataset.
- **input_vocab_size**: specifies the size of vocabulary, determining the first dimension of the embedding lookup table. `input_vocab_size = 20,000` means that the model keeps the most 20,000 most frequent words from the training dataset. A larger vocabulary captures more words and reduces out-of-vocabulary (OOV), but increases

the embedding lookup table size. On the other hand, a smaller vocabulary reduces the table size while also increases the rate of OOV. With 20,000 words, we achieved an OOV rate of 2.29% on training and 2.97% on test data, meaning nearly all words are explicitly represented.

- **max_sequence_length:** controls the maximum length of input sequences and determines the size of positional encoding matrix. We set max_sequence_length=600 based on exploratory data analysis, showing that 95% of reviews contain fewer than 600 tokens. Longer sequences increase memory consumption and computation due to quadratic scaling in attention operations. Shorter sequences may lose important context from longer reviews. The value of 600 captures adequate contextual information for accurate sentiment prediction, with reasonable memory consumption.

Dropout

Initially, we tested several models without using any dropout. These models were stuck at test accuracy around 0.84 to 0.85 and did not achieve 0.87. Only after we changed the dropout to 0.1 then we can achieve the test accuracy of 0.8664. The test accuracy with dropout = 0 is 0.8438, while with dropout of 0.2, the test accuracy is 0.8621. The corresponding notebook for these experiments is Assignment_3_v5.1-Arif-a1915248.ipynb and Assignment_3_v5.2-Arif-a1915248.ipynb respectively. Figure 6 shows the model with dropout = 0 and figure 7 displays the model with dropout = 0.2.

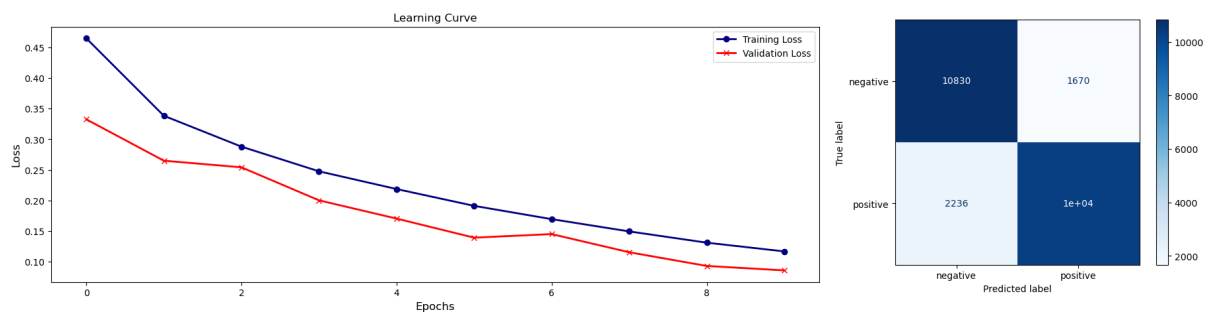


Figure 6: Learning Curve and Confusion Matrix v5.1 with Dropout = 0

Figure displays the learning curve and confusion matrix for this implementation.

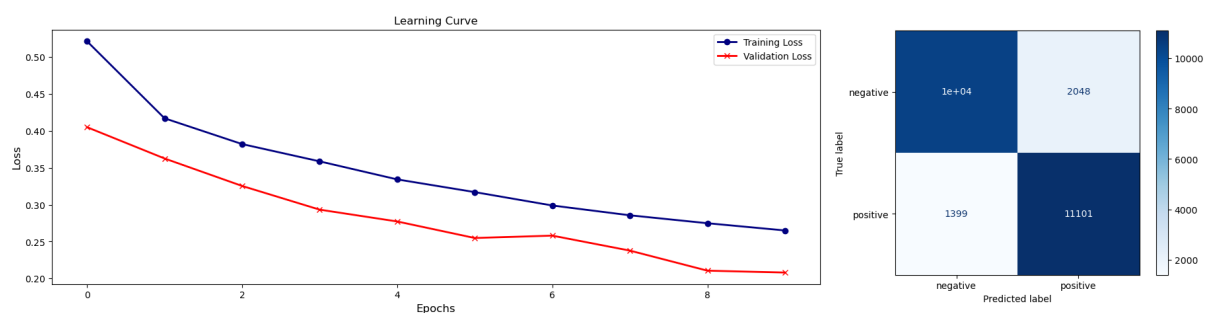


Figure 7: Learning Curve and Confusion Matrix v5.2 with Dropout = 0.2

With dropout=0.1, the test accuracy improved to 0.8664 from 0.8438 (0.0226 increase), showing that moderate regularisation enhanced generalisation. This rate forces the model to develop more robust features that do not rely on specific neuron combinations. With

dropout= 0.2, the test accuracy was 0.8621 (0.0043 decrease from 0.8664). Hence, dropout = 0.2 may over-regularise the model and offset any additional regularisation benefit. Therefore, dropout = 0.1 represents the optimal balance, providing sufficient regularisation without over-regularise the model's expressiveness.

Question 1 - Attention

Explain how the self-attention mechanism works and why it is important for sequence-based tasks.

Answer:

Self-attention is a mechanism that allows each token to attend to all other tokens in a sequence, computing a weighted representations that incorporates global information. The mechanism operates through three components: Query (Q), Key (K), and Value (V). For each token, the Q captures what information that token is seeking, the K represents what information each token (including itself) offers, and the V contains the actual information to be aggregated (Vaswani et al., 2017). The attention weight can be computed using this formula:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

QK^T computes similarity score between queries and keys. The $\sqrt{d_k}$ prevents the dot product becomes too large (extreme values of the softmax output). The softmax normalises these scores into a probability distribution. These weights then modulate the values, producing a weighted sum that emphasises relevant information and de-emphasises irrelevant tokens.

Multi-head attention extends this concept by running multiple attention operations in parallel, each with different representations of Q, K, and V. In this project, we used 4 attention heads, allowing the model to attend to information from 4 different representation subspaces. Each head captures different types of relationships.

Self-attention is crucial for sequence-based task, because review texts (in this assignment) often contain long-range dependencies. Traditional recurrent neural network architectures process sequence sequentially, gradually losing information from a distant tokens due to vanishing gradients. On the other hand, self-attention allows each token to directly attend to any other token, enabling the model to capture these long-range dependencies. This feature makes attention superior to capture sentiment-relevant pattern over the sequence, allowing the model to learn certain words that signaling sentiment.

Question 2 - Analyzing Model Performance

If your model achieves low accuracy, identify two potential reasons and suggest improvements.

Answer:

According to notebook Assignment_3_v1-Arif-a1915248.ipynb, the model achieves 0.8664 test accuracy, demonstrating good performance on the IMDB dataset. However, several factors could lead to poor model performance:

- Effects of Tokenisation

The original notebook used counting word frequencies in tokenisation, counting word frequencies across the training dataset and retaining the 20,000 most common words. We tried another tokenisation, in which we used AutoTokenizer (BERT-based-uncased) from transformers library. With the same vocabulary size and sequence length, we achieve the test accuracy of 0.8596, 0.0068 lower than the v1 model (frequency-based model). The corresponding notebook for this implementation is Assignment_3_v2-Arif-a1915248.ipynb. Figure 8 displays the learning curve and confusion matrix on testing dataset of this implementation.

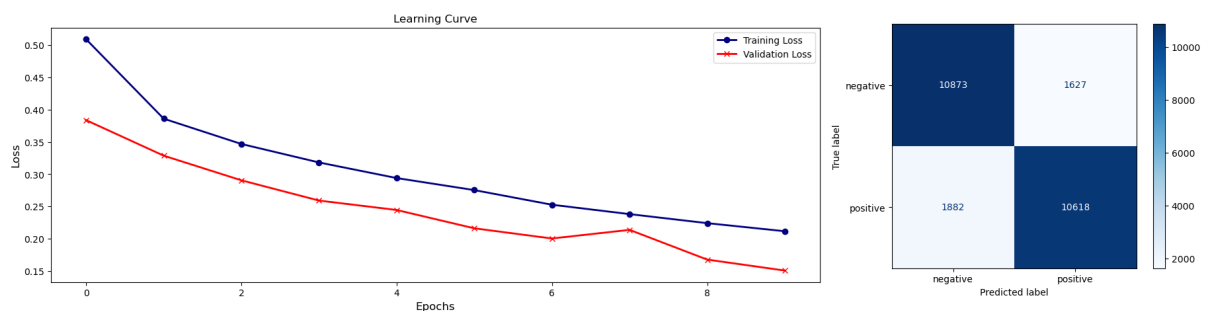


Figure 8: Learning Curve and Confusion Matrix v2

Frequency-based tokenisation performed slightly better because it aligns with the specific vocabulary pattern of the IMDB dataset. By selecting most frequent words, the tokeniser prioritise words that appear most often, which tend to be common sentiment-carrier word as well as essential functional words. These frequent words capture the essential information for sentiment classification. BERT-base-uncased tokenisation splits the words into subword, optimised for diverse task across multiple domains. This tokenisation can introduce unnecessary complexity for the model. The difference in accuracy suggests that for this specific task related to IMDB dataset, both tokenisation can capture sufficient information. On the other hand, frequency-based approach is slightly more efficient than general-usage tokenisation.

- Selection of Sequence Length

The model v1 notebook uses sequence length of 600. We tested whether a different sequence length produce a substantial difference in the result. With the same tokenisation technique and vocabulary size from model v1, we changed the sequence length to 256 (lower than model v1) and 1024 (higher than model v1). Figure 9

shows the sequence length of 256 result (test accuracy of 0.8399) and figure 10 shows the sequence length of 1024 result (test accuracy of 0.8621).

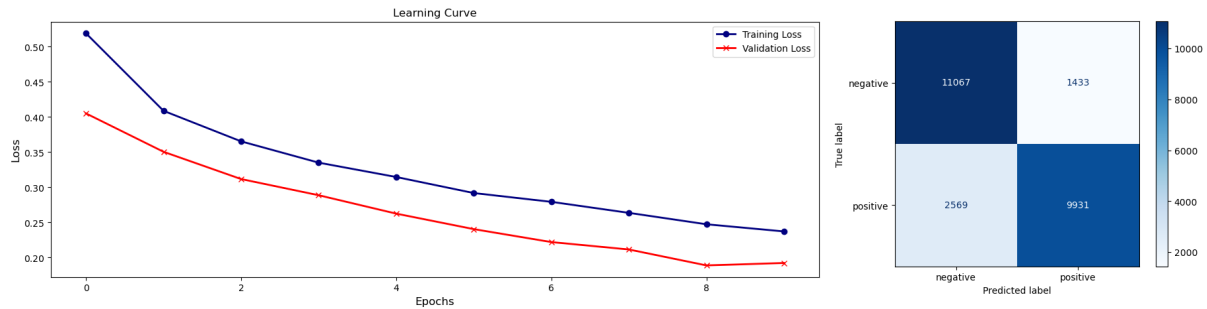


Figure 9: Learning Curve and Confusion Matrix v3.1 with Sequence Length = 256

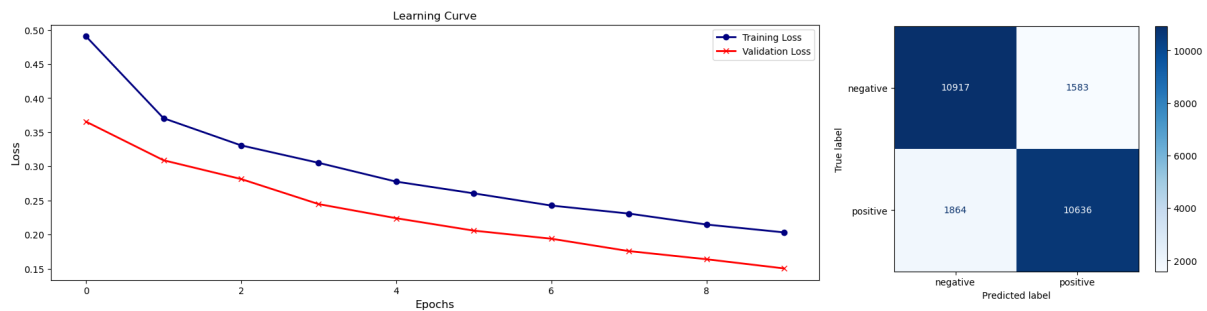


Figure 10: Learning Curve and Confusion Matrix v3.2 with Sequence Length = 1024

Although seq_length of 1024 has 424 more sequences, the test accuracy was 0.8621, 0.0043 lower than v1 model. Similarly, with seq_length of 256 (344 fewer sequences), the model achieved test accuracy of 0.8399, 0.0265 lower than v1 model, showing that important contextual information is lost when the sequence is truncated. Our EDA shows revealed that roughly 95% of IMDb reviews contain fewer than 600 tokens. We used seq_length = 600 to balance computational efficiency with adequate contextual information to make accurate predictions.

The length of the sequence affects the training time. For a full training time, seq_length = 256 can be completed in 5 minutes, seq_length = 600 in 20 minutes, and seq_length = 1024 in 52 minutes. The quadratic scaling of attention complexity ($O(n^2)$ in sequence length) explain this exponential growth in computation. Furthermore, using padding tokens beyond the 95th percentile may introduces more noise.

- Learning Rate Tuning

Using learning rate of 0.001 resulted in the test accuracy of 0.8664 and satisfy the assignment requirement. We further tried to test whether different learning rate might significantly improve the model performance. We test the same model with a learning rate of 0.01 and 0.0001. With the learning rate of 0.01, the test accuracy is 0.5, while with the learning rate of 0.0001, the test accuracy is 0.8498. The corresponding notebook for these experiments is Assignment_3_v4.1-Arif-a1915248.ipynb for learning rate of 0.01, and Assignment_3_v4.2-Arif-a1915248.ipynb for learning rate of 0.0001.

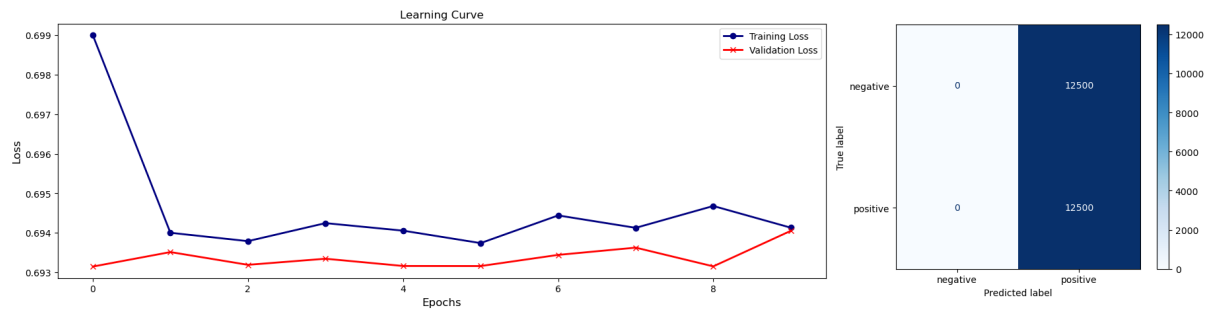


Figure 11: Learning Curve and Confusion Matrix v4.1 with Learning Rate = 0.01

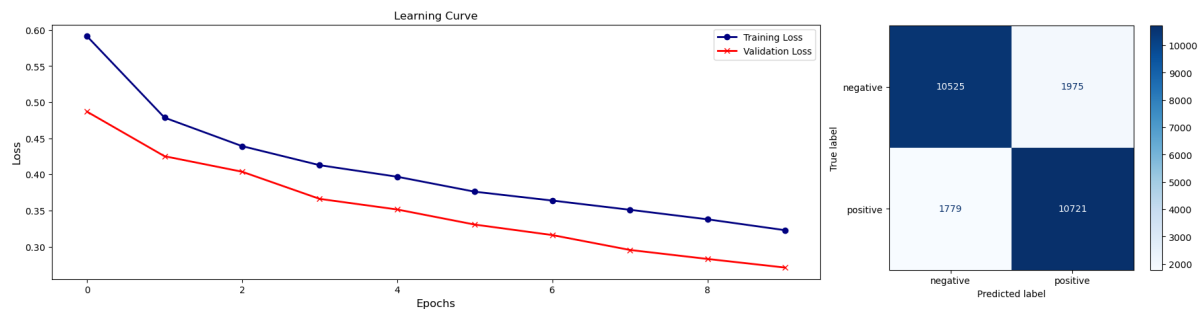


Figure 12: Learning Curve and Confusion Matrix v4.2 with Learning Rate = 0.0001

Figure 11 and figure 12 show the the results from these experiments respectively.

A learning rate of 0.01 is excessively high, causing the optimiser to take oversized steps during the training. Although Adam adapts the learning rates, maintaining excessively high learning rate causes the optimisation to diverge, de-stabilising the model's learning process. The confusion matrix from this experiment showed the model predicting only one class for nearly all examples, indicating training failure. On the other hand, learning rate of 0.0001 is too small. While the training phase remains stable, the parameter updates are small and have a slower learning progress. This will require more epochs to converge.

The learning rate of 0.001 balances the training stabilisation and convergence. This value allows Adam to make a large steps to explore the loss subspace, while remaining stable. The experiments show that learning rate is a critical hyperparameter for the model to have a stable training phase and achieve good performance.

As a summary, the model v1 performs well on the IMDB dataset, with the test accuracy of 0.8664. Several suggestions that can be explored to improve the model performance:

- Explore other regularisation technique, such as L1 or L2 regularisation.
- Implement ensemble methods to combine predictions from different models.
- Data augmentation techniques, such as paraphrasing or synonym replacement.

References

- Grefenstette, G. (1999). Tokenization. In H. van Halteren (Ed.), *Syntactic wordclass tagging* (pp. 117–133). Springer Netherlands. https://doi.org/10.1007/978-94-015-9273-4_9
- Kingma, D. P., & Ba, J. (2014). Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2017). Attention is all you need. *Advances in neural information processing systems*, 30.
- Verma, P., & Khandelwal, B. (2019). Word embeddings and its application in deep learning. *International Journal of Innovative Technology and Exploring Engineering*, 8, 337–341. <https://doi.org/10.35940/ijitee.K1343.0981119>

Acknowledgement of Generative AI Usage

In creating this report, we used Generative AI (Gemini and Claude) tools as an additional tool to brainstorm relevant topics, problem exploration, concept clarification, and language correction. The role of Generative AI was purely supportive. We confirm that the analysis in this report is authored by us.